Ring Wars

Game Design Document | Assignment 01 | First Draft

Jim Powell | Multi Agent AI for Game Development
Instructor: Joshua Burdick | Last updated: 2026-10-19

Full GDD (Markdown on GitHub)

Executive Summary

Ring Wars is a singleplayer, turn-based tactical squad shooter set in the Iris Expanse. The player commands a standard squad of **4 operators** fighting the Directorate's occupation of Marendel. The Nova resistance raids communications, power, supply routes, prisons, and command facilities, gradually turning a foothold into a campaign to liberate the planet.

Element	Design
Platforms and controls	Windows and macOS; mouse and keyboard
View	Elevated 3D battlefield, free tactical camera, and directed action shots
Audience	Players who enjoy coordinated turns, squad building, and developing familiar characters
Campaign	Authored story operations, mission choices, and generated side missions

Player experience

The squad is capable and outnumbered. The player succeeds by finding an angle, combining specialists, and preserving a way home. Valid attacks hit, with damage varying inside a visible range. Routes, cover directions, damage previews, and enemy control cones make the consequences of a plan readable.

An Assault Rifle operator can cover the approach a Shock Sword operator needs. A Sniper can remove a distant threat, while a Medic keeps the team moving. Destroying cover changes a firing position; completing an objective can turn a raid into a pressured withdrawal. The next decision follows from what just happened on the battlefield.

Between operations, headquarters connects the War Table, Barracks, and Armoury. The player chooses missions, equips the squad, trains abilities, and develops recognizable operators through callsigns, appearance, and specialization. Captured weapons and rescued specialists expand those choices through the campaign's events.

Victory and defeat

The campaign culminates in defeating occupation command in the capital. A mission succeeds when its objective is completed and the squad extracts, or when the defending force is defeated and the location secured, as required by that operation.

If the whole squad is Downed, the mission fails and remains available for another attempt. During a successful extraction, operators left outside the extraction zone are lost. Bringing an operator home therefore matters alongside completing the job.

Developed by Jim Powell with AI assistance.

Game Mechanics

Core loop

1. **Choose an operation.** At the War Table, consider the objective, conditions, and rewards of story and side missions.
2. **Prepare the squad.** Select operators, assign kits and gear, and train abilities that support the job.
3. **Execute coordinated turns.** Read the battlefield, choose routes, and spend actions across the squad to reach and complete the objective.
4. **Resolve the operation.** Extract under reinforcement pressure or secure the site, according to the mission's ending.
5. **Develop the resistance.** Return to headquarters, use earned equipment and credits, spend experience, and prepare for the opportunities the operation opened.

Actions and tactical consequences

Combat alternates between **Our Turn** and **Their Turn**. During Our Turn, the player can switch freely among operators and spend action points (AP) on movement, attacks, abilities, revival, and objective interactions. Moving, attacking, and moving again are valid choices. End Turn commits the squad to the enemy response.

Movement previews show routes and AP costs. Directional cover absorbs incoming damage from the protected angle, while a flank can bypass it. Armor protects the unit itself. Destructible cover can become exposed during a firefight, creating a new opportunity or making a previously useful position dangerous.

Overwatch threatens an area with reaction fire. Suppressing Fire can stop movement through its cone. The player can counter a defended lane by changing angle, breaking sightlines, using smoke, or attacking the watcher. Enemy grenades mark a delayed blast area, giving the player a turn to reposition at the cost of changing the plan.

Example of play

In **Supply Sabotage**, the squad must plant charges on a Directorate stockpile and extract. A sentry covers the direct approach. The player inspects its cone, sends the Shock Sword operator along a longer side route, and uses the Rifle operator to cover movement. The Sniper attacks an exposed defender; the visible damage result helps the player decide whether to finish that target or advance.

After the charges are planted, reinforcements make withdrawal urgent. An optional cache offers a reward but adds distance and exposure. The player chooses whether to take that detour, uses smoke to conceal a crossing, and keeps the Medic close enough to help a Downed teammate. Success rewards both the attack plan and the route preserved for getting home.

Authored operations carry the resistance story. Generated side missions combine authored spatial blocks, objectives, and enemy placements into varied tactical sites. Each offered mission retains its setup across retries, so the player can apply what the previous attempt taught them.

AI Architecture

Specialized development agents turn prose briefs into content and checked changes. Their work becomes the places, performances, encounters, and interactions the player experiences. Unity executes the tactical rules, enemy behavior, and generated mission assembly.

Voiceover Agent

When approved dialogue changes, this agent generates and integrates spoken lines so the player hears the mission briefing or exchange with matching subtitles.

Input: Approved dialogue, speaker identity, and delivery direction. **Structured output:** Line identifiers, clip references, subtitle associations, durations, and playback findings.

Level Designer Agent

When a mission brief is ready, this agent designs routes, objectives, cover, and visual references so the player has useful choices about approaching the target and withdrawing.

Input: Mission purpose, setting, enemy roles, insertion, and extraction needs. **Structured output:** Layout and brief references, route and objective annotations, fixed-camera reference views, and layout checks.

Level Builder Agent

When a level design is approved, this agent sources assets and builds the playable location so the player can move, fight, use cover, and complete objectives inside a coherent Marendel facility.

Input: Approved layout, visual references, and asset sources. **Structured output:** Source asset references, Blender and Unity deliverables, and import, navigation, cover, lighting, and playable-path findings.

Balance Agent

When a kit, encounter, reward, or playtest finding needs adjustment, this agent revises the relevant definitions so the player has useful squad choices and encounters that reward coordinated action.

Input: A specific gameplay issue and its intended experience. **Structured output:** Changed definition identifiers, the change set, and before-and-after gameplay observations.

Testing Agent

When a playable change is ready, this agent checks the affected path and reports reproducible failures so the player can deploy, complete the mission, extract, and use the rewards.

Input: The changed feature and expected player-visible behavior. **Structured output:** Expected and observed results, pass or fail status, reproduction steps, and evidence.

One Wow agent

The **Level Builder Agent** produces the most visible result: a playable Marendel location. In Supply Sabotage, the player must recognize the stockpile, read a protected approach, find useful firing positions, and see a route to extraction. Lighting, materials, destructible cover, and surrounding scenery make those decisions part of a believable occupied facility.

Technical Strategy

Foundation and production scope

Ring Wars uses **Unity 6 URP**, C# gameplay systems, editable content definitions, and Blender. The eight-week course builds on the existing game through playable mission slices, connecting the brief, environment, encounters, dialogue, objectives, and return to headquarters.

Agent contracts and handoffs

Each agent accepts a prose brief and returns a **JSON result manifest** identifying the task, agent, status, summary, inputs, deliverables, checks, and open questions. Native files remain the working artifacts. The Level Designer's package feeds the Level Builder; Voiceover uses approved dialogue; Balance and Testing evaluate the playable result. Jim directs priorities and reviews visual quality, timing, and feel.

Biggest technical constraint

Art production is the main constraint. Environments and operators need a consistent look while supporting readable cover, destruction, navigation, and close camera views. Existing licensed asset packs, shared Blender libraries, modular pieces, and the asset generation pipeline supply reusable foundations. Checked tactical layouts become blockouts and fixed-camera references, then assembled Unity locations. Generated concepts, selected textures, and voice performances support this process. Jim reviews the result from tactical and action cameras.

Token budget

The initial estimate is **1,000,000 text-agent tokens per week**, or **8,000,000 over eight weeks**. A reference production cycle budgets:

Agent task	Tokens
Voiceover preparation and integration	15,000
Mission layout and reference pass	80,000
Level build and Unity integration	160,000
Focused balance pass	35,000
Mission and campaign-path verification	30,000
Total per cycle	**320,000**

Two cycles use 640,000 tokens, leaving 360,000 weekly for integration and rework. A cycle can revise an existing mission. Estimates include input, output, and reported reasoning across calls; initial usage will refine them. Image and speech costs are tracked separately. Gameplay uses **zero generative-AI tokens per mission**.

API constraints

Text agents share the connected account's model and project rate limits; focused context and bounded concurrency keep tasks within request, token, and context limits. Structured results are checked before handoff. Speech generation runs by line within ElevenLabs credits and concurrency limits, reusing unchanged performances. Image generation uses focused batches and retained references to manage quota and turnaround. Imported media and authored assets are used by the game.

API references: OpenAI rate limits, structured outputs, and ElevenLabs API errors.